

AI.OPEN Usage für Datenschutz Konforme State of the Art AI bei OPEN

<https://ai.open.de> bietet uns ein Middleware den uns folgende State of the Art Modellen (stand 12.06.2026) zu verfügung stellt:

Models						
This page is a read-only view of configured language models for this instance.						
Read-only view Models are configured in LiteLLM's config.yaml. Edit it on the host and restart Exulu to pick up changes. Use LiteLLM's own Admin UI for runtime keys, budgets, and access groups. Open LiteLLM Admin UI						
Model name	Context (in / out)	Modalities	Cost (in / out)		Active	Tags
 EU vertex-gemini-3.5-flash	1.0M / 66K	vision pdf audio tools	\$1.50/M	\$9.00/M	Active	
 EU gemini-3.1-flash-lite	1.0M / 66K	vision pdf audio tools	\$0.25/M	\$1.50/M	Active	
 EU qwen3-235b	262K / 16K	tools	\$0.25/M	\$1.00/M	Active	
 EU vertex-gemini-2.5-pro	1.0M / 66K	vision pdf audio tools	\$1.25/M	\$10.00/M	Active	
 EU vertex-gemini-2.5-flash	1.0M / 66K	vision pdf tools	\$0.30/M	\$2.50/M	Active	
 EU claude-opus-4-7	1.0M / 128K	vision pdf tools	\$5.00/M	\$25.00/M	Active	
 EU claude-sonnet-4-6	1.0M / 64K	vision pdf tools	\$3.00/M	\$15.00/M	Active	
 EU claude-haiku-4-5	200K / 8K	vision pdf tools	\$1.00/M	\$5.00/M	Active	

Diese models laufen über den Google Cloud Platform (GCP) umgebung von OPEN. Benutzer werden in der Middleware Teams zugeordnet (ADX, CX, SC), was dafür sorgt das automatisch anfragen an den sprach modellen ueber den jeweiligen GCP Projekt der Unit geleitet wird. Das jeweilige GCP Projekt wird pro Unit durch folgende personen gemanaged:

- CX - Elias Dreisbach
- ADX - Jens Parree
- SC - Andreas Hucks

Authentifizierung auf <https://ai.open.de> erfolgt mittels Google SSO.

Verwendung von Models mittels OPENAI Kompatibele Endpunkten

Die meisten anwendungen, IDE's, Coding Agents etc.. nutzen OPENAI Kompatibele Endpunkten für die einbindung von Sprachmodellen.

Der OPENAI Kompatibele Endpunkt auf ai.open ist wie folgt:

https://backend.ai.open.de//litellm/:project/v1/*

:project muss hier ersetzt werden mit entweder "DEFAULT" falls kein Projekt zugeordnet werden soll, oder der Projekt ID (meistens der Kunde für den das Sprachmodell verwendet wird). Projekten anlegen, und der ID kopieren erfolgt auf <https://ai.open.de/projects>.

Die authentifizierung beim API erfolgt wie folgt.

1. Persönliches token, ein user bezogenes Token was über <https://ai.open.de/token> zu verfügung gestellt wird, und mittels "Authorization" header im Format "Bearer <token>" verschickt wird.
2. API Key, wird über <https://ai.open.de/keys> generiert, und im header "exulu-api-key" mitgeschickt

Verwendung mit Claude Code

Um Claude Code via den ai.open Gateway zu verwenden muss folgende settings.json in der .claude folder angelegt werden (meistens im Root vom Repository wo du gerade arbeitest).

Bei model, und availableModels können im Prinzip alle Models gelistet werden die ai.open bereitstellt, also auch:

- vertex-gemini-3.5-flash
- qwen3-235b
- gemini-3.1-flash-lite
- vertex-gemini-2.5-flash
- vertex-gemini-2.5-pro

Im ANTHROPIC_BASE_URL, hier muss den <PROJECT_ID> mit entweder "DEFAULT" oder den Project ID aus <https://ai.open.de/projects> ersetzt werden.

Bei apiKeyHelper muss <MY_TOKEN> ersetzt werden mit dein <https://ai.open.de/token> token. Sonst Funktioniert CLaude Code genau gleich wie wenn mann es direkt ueber die Anthropic Server authentifiziert.

```
{  
  "model": "claude-opus-4-7",  
  "availableModels": [  

```

```

        "claude-sonnet-4-6",
        "claude-haiku-4-5",
        "claude-opus-4-7"
    ],
    "env": {
        "ANTHROPIC_BASE_URL": "https://backend.ai.open.de/litellm/<PROJECT_ID>",
        "CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS": 1,
        "DISABLE_AUTOUPDATER": 0
    },
    "apiKeyHelper": "echo <MY_TOKEN>"
}

```

Continue.dev + ai.open

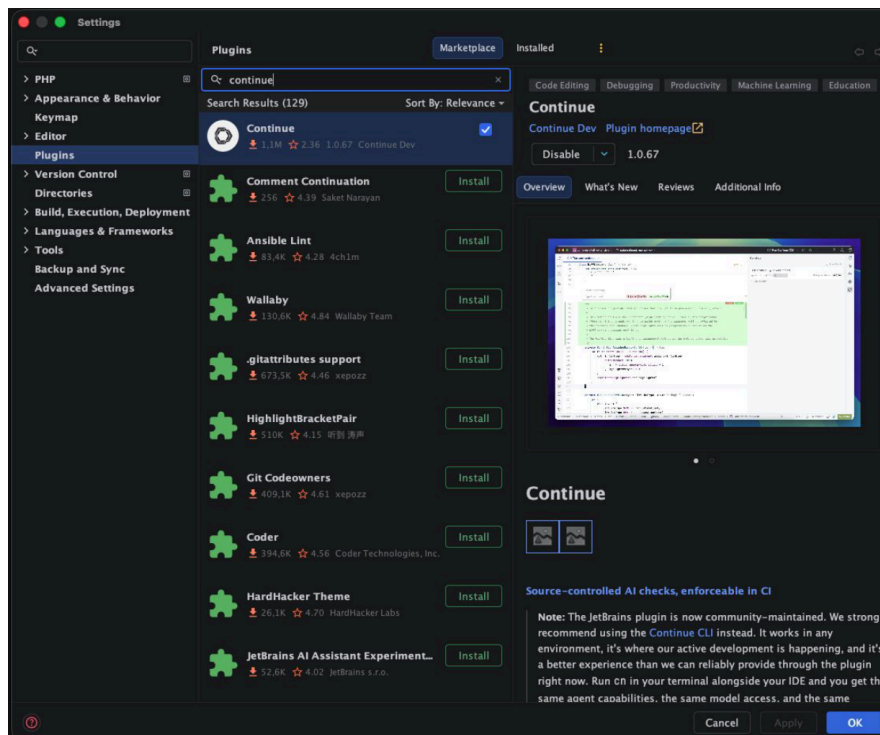


Um continue.dev zu konfigurieren kannst du folgende anleitung folgen

So installiert ihr das "continue" Plugin in PhpStorm:

- Öffnet Settings (Cmd+, / Ctrl+Alt+S)
- Navigiert zu Plugins im linken Menü
- Sucht im Marketplace nach "Continue"
- Klickt auf Install
- Startet die IDE ggf. neu, um das Plugin zu aktivieren

Das Plugin erscheint anschließend in der rechten Sidebar oder im "Tools" Menü.

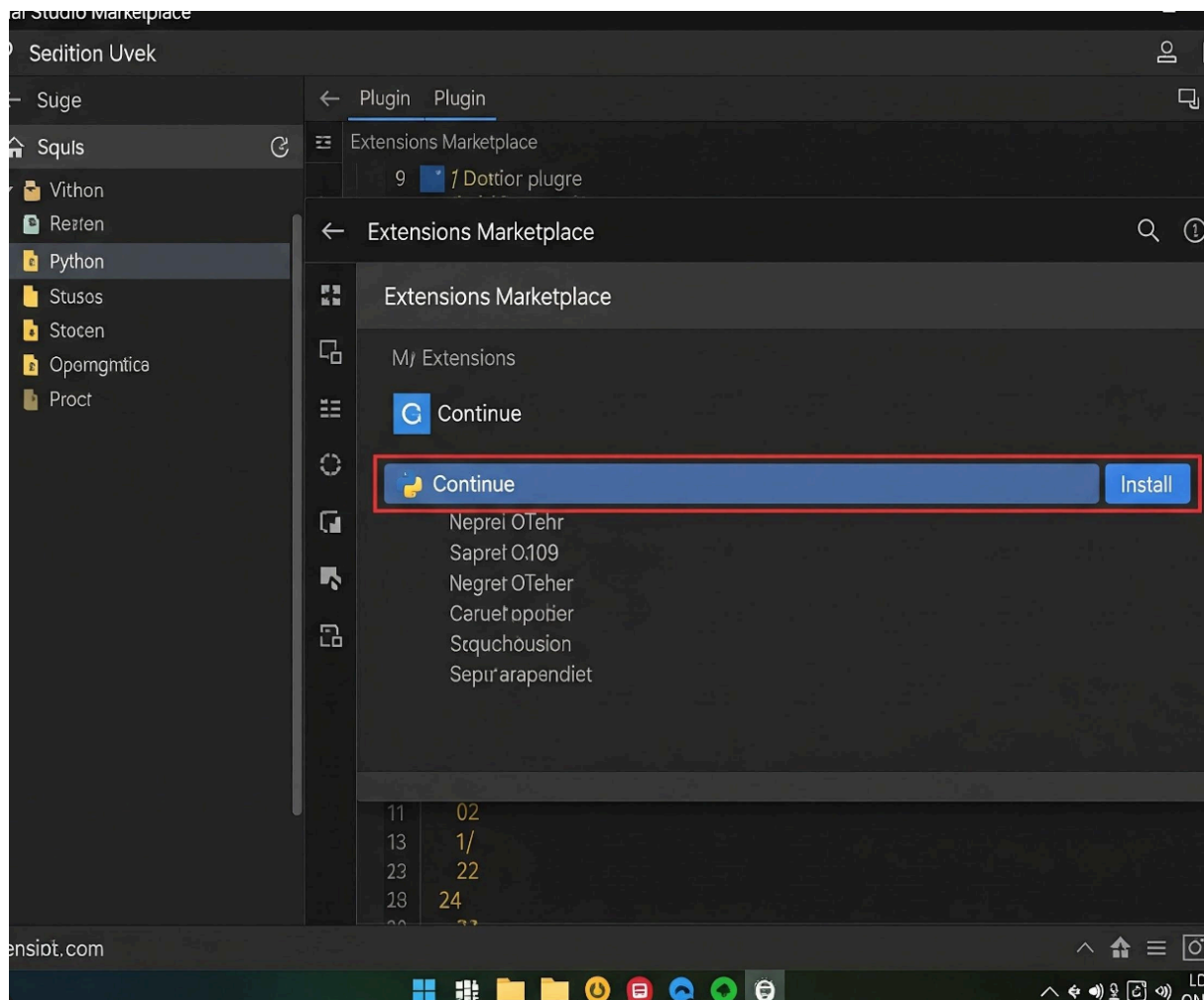


Continue Installation in VS Code

So installiert ihr die Extension in Visual Studio Code:

- Öffnet die Extensions-Ansicht (Cmd+Shift+X / Ctrl+Shift+X)
- Sucht nach "Continue" im Marketplace
- Klickt auf Install
- Das Continue-Icon erscheint nun in der linken Aktivitätsleiste

Nach der Installation könnt ihr direkt mit der Konfiguration eurer AI.OPEN Tokens starten.



Zugriff auf die config.yaml

So öffnet ihr die Konfigurationsdatei in VS Code:

- Klickt auf das Continue-Icon rechts Oben
- Klickt oben im Continue-Fenster auf das Zahnrad-Icon (Settings)
- Wählt links den Punkt "Configs" aus
- Klickt hinter dem Eintrag Local Config auf das Zahnrad-Icon

In dieser Datei hinterlegt ihr später euren AI.OPEN Token und definiert die Modell-Provider.

Tragt die folgenden Zeilen in eure config.yaml ein.

Ersetzt dabei TOKEN durch den zuvor kopierten Token von ai.open.de.

Nach dem Speichern erkennt die IDE den neuen Provider automatisch.

```
name: Local Config
version: 1.0.0
schema: v1
models:
```

- name: AI.OPEN
provider: openai
apiBase:
https://backend.ai.open.de/litellm/<PROJECT_ID>/v1
apiKey: <TOKEN>
model: AUTODETECT
roles:
- chat
- edit
- apply
capabilities:
- tool_use
- image_input

Happy Coding mit continue

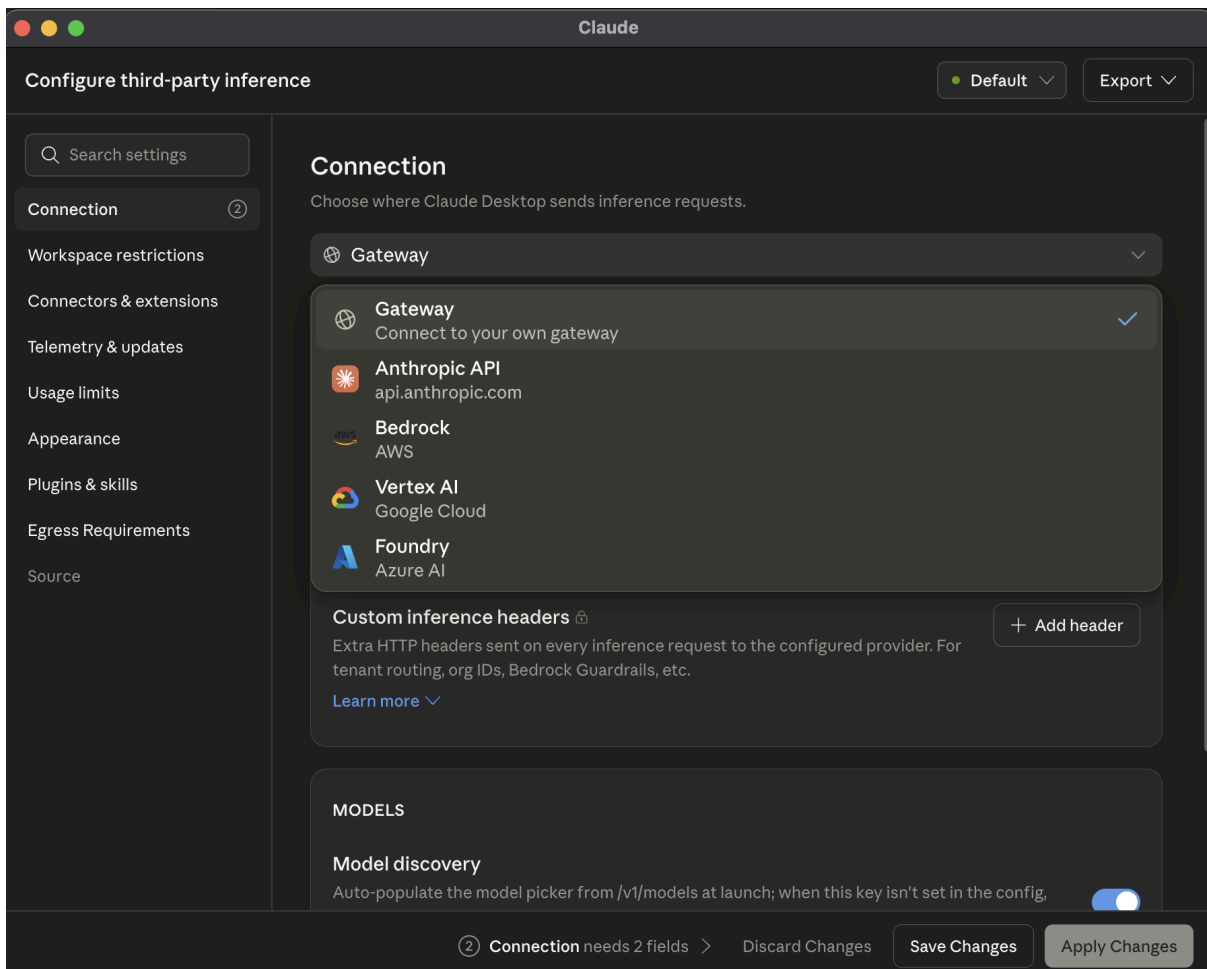
7

Nun sollten alle Projekte als Modelle in
continue angezeigt werden.
Die Modell-Namen sind immer so
aufgebaut: Projekt/Agent
Dadurch wird der Token-Verbrauch auf
die jeweiligen Projekte geloggt.

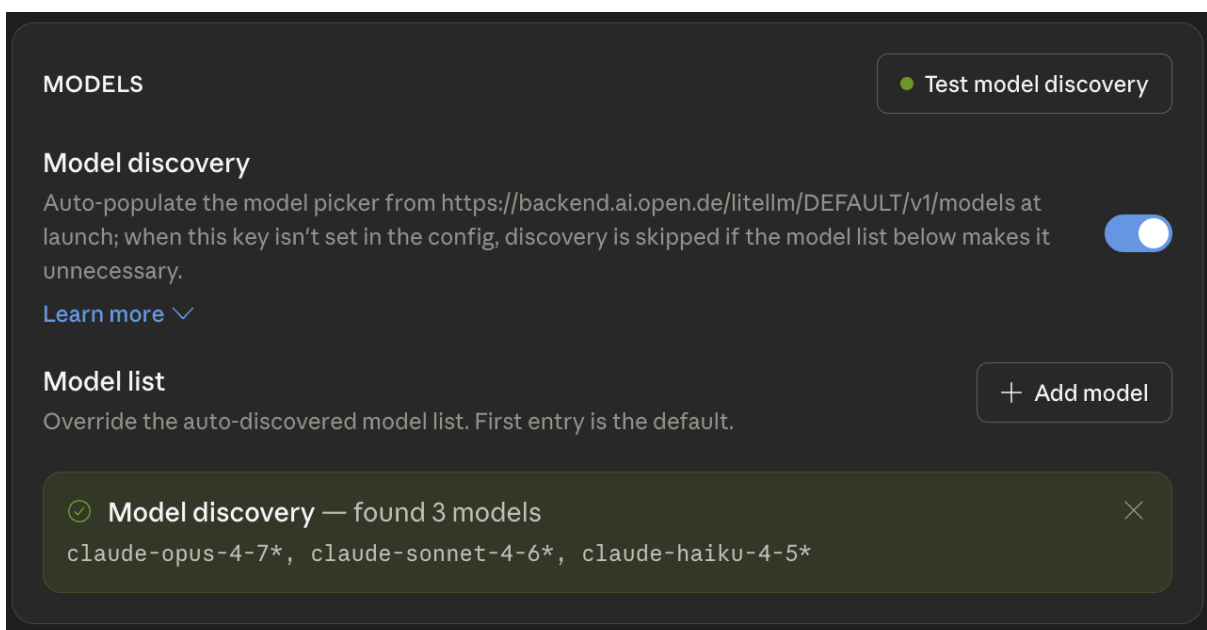
Claude Desktop Client nutzen mit ai.open

Um die Claude Desktop app (<https://claude.com/download>) mit ai.open, und somit
datenschutz Konform und mit Budget zuordnung zu nutzen folgende schritten machen:

1. Öffne die CLaude Desktop app
2. Auf "Help" in der System Menu Leiste -> Troubleshooting -> "Enable Developer Mode"
3. Claude Desktop App macht jetzt ein Restart
4. Es gibt jetzt im System Menu LEiste ein neuer Punkt "Developer" -> Configure Third Party Inference



5. Hier muss "Gateway ausgewählt werden
6. Credential Kind "Static API Key"
7. Gateway base url https://backend.ai.open.de/litellm/<PROJECT_ID>
8. Gateway API Key dein <https://ai.open.de/token> token
9. Click auf "test model discovery", sollte so aussehen:



10. "Apply Changes"

11. Viel Spaß

Direkte nutzung ai.open Agents

Im ai.open werden eigene Custom Agents entwickelt und zu verfügung gestellt fuer OPEN Teams durch as KI team.

Aktuell verfügbare Agents:

1. OPEN PULSE: Kunden Analyse (Haupt Ansprechpartner Robert Schaperjahn)

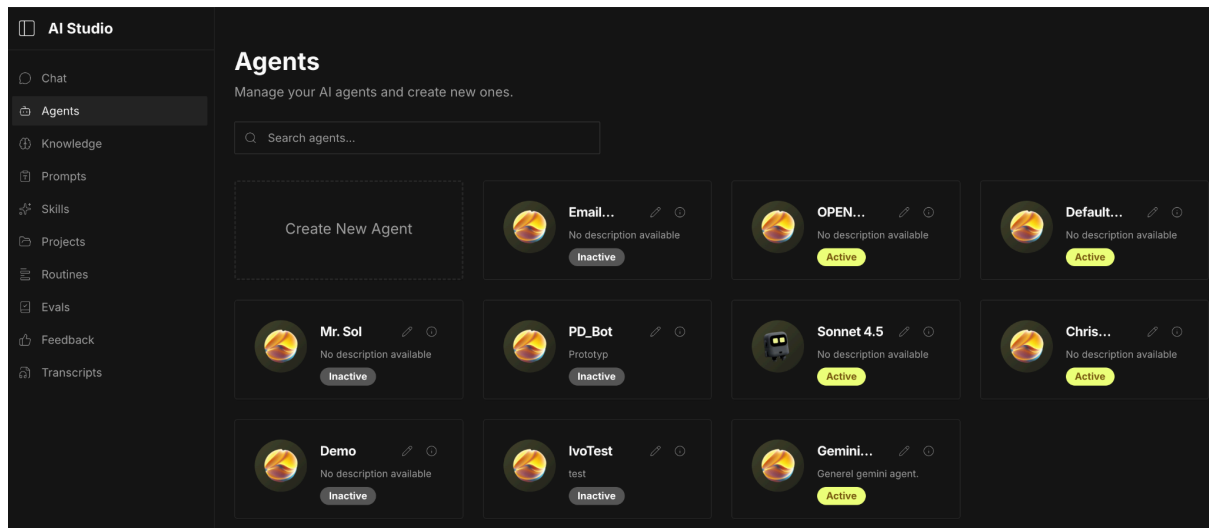
<https://ai.open.de/agents>

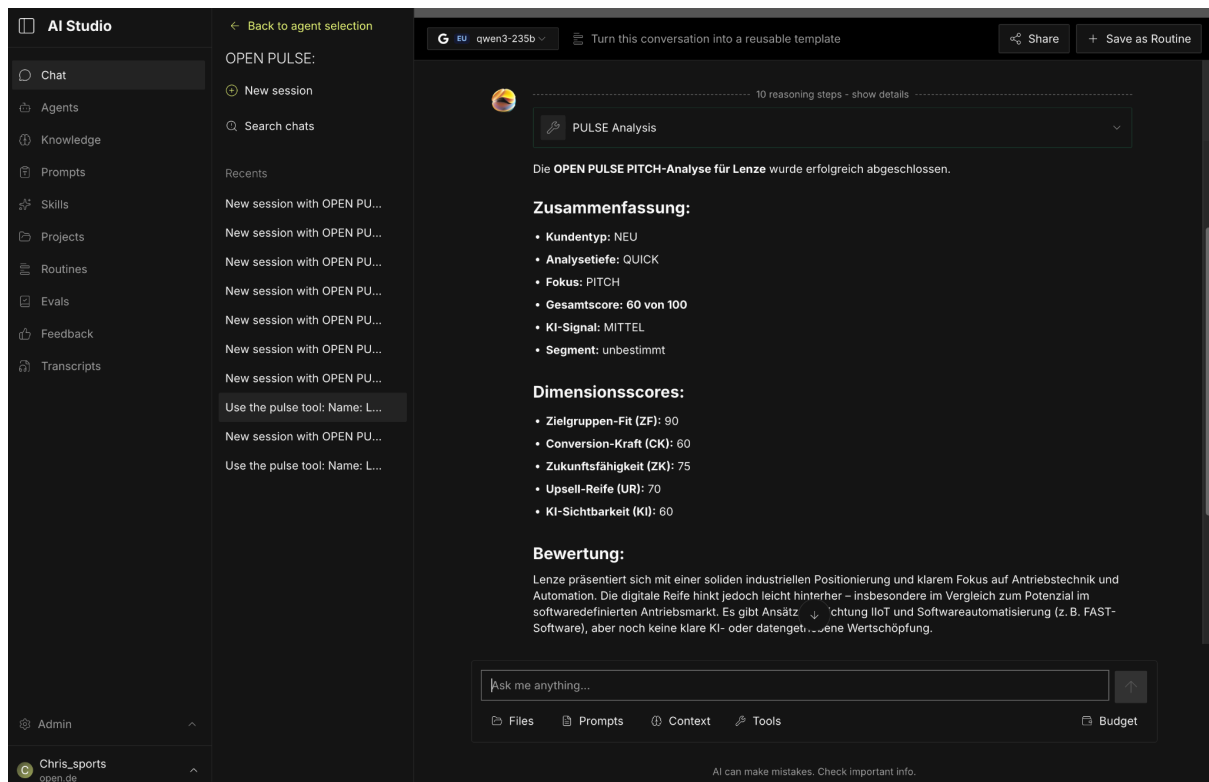
Agents nutzen ein Kombination aus sprachmodell, wissensquellen die in ai.open konfiguriert sind und integrationen mit andere systemen um bestimmte use cases wie z.b. Angebots generierung, Kunden analyse oder call Transkripten abzubilden.

Das KI-Team pflegt ein Roadmap an KI Agenten die auf Ai.OPEN verfgubar werden in die nahe zukunft, u.a.:

1. Ausschreibungs Agent
2. Projekt Management Agent
3. Marketing Support Agent

Falls es ideen zu weiteren agenten gibt, oder ihr wollts mit entwickeln, schick gerne ein nachricht an Daniel Claessen oder Michael Möckel.





Token Budgets

Jeder Benutzer innerhalb OPEN hat per default 20 USD Token Budget im Monat. Jeder Benutzer kann seinen aktuellen Verbrauch auf ai.open in der Menu Leiste sehen.

Admins und Team leads können auf <https://ai.open.de/budgets> individuell pro Benutzer Budgets erhöhen. Wichtig, wir haben OPEN derzeit ein max monatliches Budget von 3.500 USD, das System blockiert automatisch, wenn wir darüber gehen. Eine Erhöhung vom Gesamtbudget kann nur im KI Team entschieden werden, innerhalb der Teams können aber die KI-Leads für die jeweiligen User Budgets anpassen.

 **Budgets**

Set spend budgets per user, role, team, project, or agent. Budgets are enforced by LiteLLM on the entity's usage.

Entity budgets

Select an entity type, then set individual budgets. Select multiple rows to apply a budget to all of them at once.

UsersRolesTeamsProjectsAgents

☒ Global per-user budget

Amount

Reset

Monthly

☒ Show budget status to user (in chat).

 Save

☐	User	Budget	Actions
☐	Maksymilian	\$0 / \$200%	 Edit
☐	Thomas	\$0 / \$200%	 Edit
☐	exulu	\$0 / \$200%	 Edit
☐	chris_sports@open.de	\$0 / \$200%	 Edit

Controlling

Super admins haben zugriff auf das Controlling backend auf <https://backend.ai.open.de/litellm-admin/ui/>. Hier kann pro projekt, benutzer, rolle und team daten exportiert werden die die genau zuordnung des Token verbrauchs fur Controlling zwecken ermöglicht.

Alle aufrufen via ai.open werden automatisch mit Projekt, Rolle, Benutzer und Team tags versehen, diese nachher das Filtern der Kosten zuordnung erlaubt. Fur den Export und Filter möglichkeiten auf https://backend.ai.open.de/litellm-admin/ui/?page=new_usage gehen -> Usage View auf "Tag Usage" wechseln, und entweder exportieren oder nach spezifischen Tags vorfiltern. Wichtig, durch ein bug im Litellm proxy werden bei den Tag ansicht die Kosten doppelt gezahlt weil die gleichen Kosten mit mehreren Tags (z.b. Projekt + User Tag) gerechnet werden, dies ist im CSV Export zu berücksichtigen.

LiteLLM

v1.85.1

DocsBlogUser

AI GATEWAY

Virtual Keys

Playground

Models + Endpoints

Agentic

MCP Servers

Skills

Guardrails

Policies

Tools

OBSERVABILITY

Usage

Logs

Guardrails Monitor

ACCESS CONTROL

Teams

Internal Users

Organizations

Access Groups

Budgets

DEVELOPER TOOLS

API Reference

AI Hub

Learning Resources

Usage View

Select the usage data you want to view

Global Usage

Team Usage

Customer Usage

Tag Usage

Agent Usage (A2A)

User Usage

Select Time Range

5 Jun, 13:57 - 12 Jun, 13:57

Filter by user

Select user to filter...

Cost

Model Activity

Key Activity

Project Spend Jun 5 - Jun 12, 2024

Total Spend

\$272.8207

Max Budget

No limit

Ask AI

Export Data

Usage Metrics

Total Requests

6,567

Successful Requests

5,290

Failed Requests

1,277

Average Cost per Request

\$0.0415

Total Tokens

370,683,248

Daily Spend

\$80

\$60

\$40

\$20

\$0

2026-06-04

2026-06-05

2026-06-07

2026-06-08

2026-06-09

2026-06-10

2026-06-11

2026-06-12